

Object Oriented Programming (OOP)

Semester: 2nd

Credit Hour: (01)

Department of Computer Science

Course Instructor: Engr. Zakria Bacha

Lab Task No. 5

Part II Structure in C++

Previous Lab:

In previous lab, we have covered below three topics:

1. Structure
2. Array of Structure
3. Nested Structure

Current Topics for Lab:

1. Pointer to Structure
2. Accessing structure with pointers
3. Pointer as Structure Member
4. Array as structure Member
5. Passing Structure Variable to Function

Pointer to Structures:

1. A structure variable has an address
2. Pointers can be used to point to structure variables.
3. Pointers to structures are variables that can hold the address of a structure variable.
4. `Student *stuPtr;`
5. The **stuPtr** pointer can point at variables of the type **Student**.

Accessing Structure with Pointers:

```
2 struct Square{
3     int width;
4     int height;
5
6 };
7
8 int main()
9 {
10     Square square_1 = {20,30};
11     // Pointer to Structur
12     Square *ptr_to_square = &square_1;
13
14     return 0;
15 }
```

Figure 1

How to access the structure members (using pointer)?

1st Method:

Use dereferencing operator (*) with dot operator (.)

Dereference Operator (*) is used with structure pointer variable; not with member variable.

```
int main()
{
    Square square_1 = {20,30};
    // Pointer to Structur
    Square *ptr_to_square = &square_1;

    cout<<(*ptr_to_square).width;
    cout<<(*ptr_to_square).height;
```

Figure 2

2nd Method:

Is there some easier way to do this?

Yes, Use arrow operator (->) instead of * and. Operator

```
9  int main()  
10 {  
11     Square square_1 = {20,30};  
12     // Pointer to Structure  
13     Square *ptr_to_square = &square_1;  
14     // Use Arrow Operator  
15     cout<<ptr_to_square->width;  
16     cout<<ptr_to_square->height;
```

Figure 3

Pointer as Structure Member?

Pointers can also be a member of structure.

```
1  #include<iostream>  
2  using namespace std;  
3  struct Square{  
4      int *width;  
5      int height;  
6  
7  };
```

Figure 4

Dereference Operator (*) is applied on structure member width variable. (.) has higher precedence.

```
9 int main()
10 {
11     int width_value = 20;
12     Square square_1 = {&width_value, 30};
13     // Use Arrow Operator
14     cout<<*square_1.width;
15     cout<<square_1.height;
16
17     return 0;
```

Figure 5

- Pointers can also be a member of a structure

```
struct Rectangle {
    Rectangle *ptr;
    int height;
};

void main( )
{
    int w = 3;
    Rectangle rect1, rect2;
    rect1.height = 4;
    rect1.ptr = &rect2;
}
```

Object Oriented Programming-Spring 23

Address 0x1003

Rect1
ptr = 0x1032
height = 4

Address 0x1032

Rect2
ptr =
height =

Figure 6

Structure as a Function Argument:

May pass **members** of **struct variables** to functions:

//function definition

```
float computeGPA(float gpa){
```

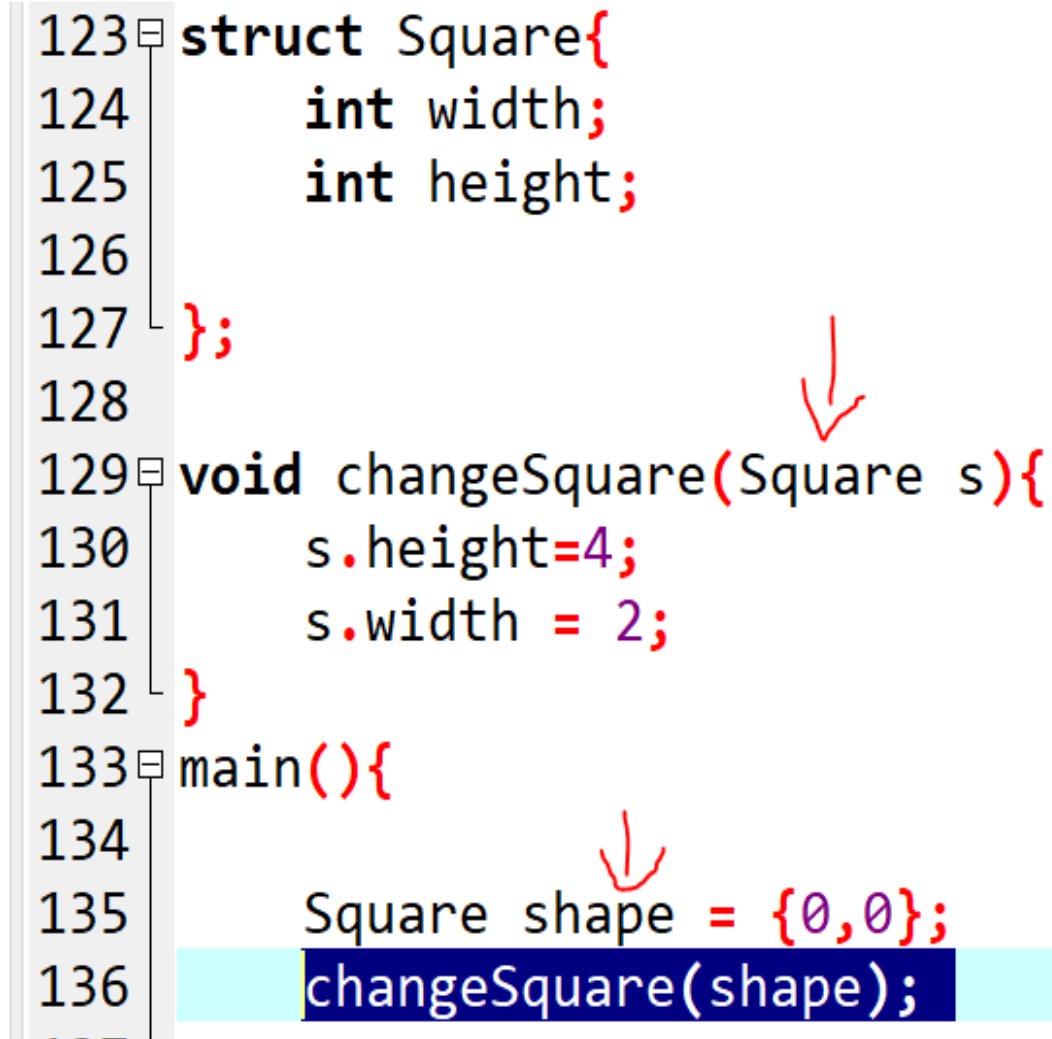
```
.....
```

```
}
```

//function call

```
computeGPA(s1.gpa);
```

May Pass Structure Variable or Object to Function.



```
123 struct Square{
124     int width;
125     int height;
126
127 };
128
129 void changeSquare(Square s){
130     s.height=4;
131     s.width = 2;
132 }
133 main(){
134
135     Square shape = {0,0};
136     changeSquare(shape);
```

Figure 7

Lab Task No.1

► Create a single structure to store student information, including name, ID, subject count, and an array of scores for five subjects. In the main () function, use a pointer to the student structure (with the arrow operator ->) for all data input and access.

Create one function, processPersonData(), to calculate the average score and display the results for the student. Also display the second-highest score. Scores cannot be less than 0 or greater than 100.

Note: There is a total of 5 subjects. The user will first enter the subject count, which must be greater than 0 and less than or equal to 5.

Input:

Enter Name: Ali

Enter ID: 101

Enter Number of Scores (0-5): 3

Enter Scores: 85 90 78

Output:

Person Record

=====

Name: Ali

ID: 101

Grades: 85.00 90.00 78.00

Average: 84.33

Second Highest Score: 85.00

=====

Lab Task No.2

Write a C program for an HR Payroll & Incentive Management System. The system will store employee information using a structure that contains **normal variables, an array as a structure member, and a pointer member inside the structure.**

In the main() function, use a pointer to structure and access all members using the arrow operator (**=>**).

Do the Task step by step:

I. Declare a structure which contain the following below members.

1. `employeeName[50]`
2. `jobTitle[30]`
3. `employeeCode`
4. `monthCount`
5. `monthlySalary[5]` (*array as structure member*)
6. `float *base_address;` (*pointer member inside structure*)

II. Also follow the below instruction for using pointer with structure.

1. Declare a structure variable. Example `Emp e ;`
2. Declare a pointer to that structure. Example `Emp *ptr_to_e = &e;`
3. Use only **=>** operator in main() for accessing structure members
4. Store the base address of array (**monthly salary**) in `base_address` pointer. (**Optional You can show the values of array**)

1. In main function ()

1. User will enter employee details.
2. Take number of months (1 to 5 only).
3. User will enter monthly salary (must be positive values).
4. User will enter extra amount. (any amount can be entered)

III. Define only one function.

1. Function 1 calculate display Payroll()

1. Calculate average monthly salary of employee.
2. Find highest monthly salary.
3. Update average salary by adding extra amount.
4. **Show the following on output:**
 1. Employee Name
 2. Job Title
 3. Employee Code
 4. All monthly salaries
 5. Average salary
 6. Highest salary
 7. Final Average salary with (Add extra Amount)

Your program should follow the following instruction.

1. Month count must be between 1 and 5.
2. Salary cannot be negative.
3. Use proper formatting (2 decimal places).
4. Use structure pointer throughout the program.

National University of Computer and Emerging Sciences
Peshawar Campus